TUGAS TERSTRUKTUR

LAPORAN PRAKTIKUM *STORE PROCEDURES*

Disusun Untuk Memenuhi Tugas Pertemuan 4

Mata Kuliah :Prak Rekayasa dan Arsitektur Basis Data

Dosen Pengampu:

Cecep Kurnia Sastradipraja, S.Kom., M.Kom.

Disusun Oleh:

Pirda Nur Awaliah 2507066

Sistem Informasi B

JURUSAN ILMU KOMPUTER

PROGRAM STUDI SISTEM INFORMASI

INSTITUT TEKNOLOGI GARUT

2026

Tugas 1 – Membuat Database dan Tabel

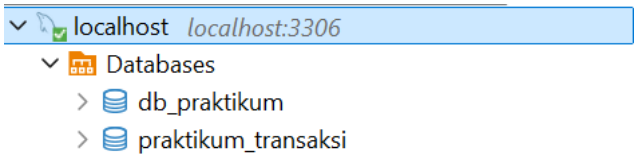
1. **Membuat Database dan dan Mengaktifkan Context**

   a) CREATE DATABASE db_distribusi; membuat wadah utama basis data perusahaan.

   CREATE DATABASE db_distribusi;

   b) USE db_distribusi; memastikan seluruh sintaks tabel dan *query* selanjutnya dijalankan di dalam database db_distribusi.

   USE db_distribusi;



2. **Membuat Tabel Barang**

   a) Kolom kode_barang dijadikan **Primary Key** sebagai identitas unik setiap produk.

```
CREATE TABLE barang (
    kode_barang CHAR(5) PRIMARY KEY,
    nama_barang VARCHAR(50) NOT NULL,
    kategori VARCHAR(30) NOT NULL,
    harga INT NOT NULL,
    stok INT NOT NULL
);
```

3. **Membuat Tabel Penjualan**

   Kolom no_transaksi ditetapkan sebagai Primary Key.

   Kolom kode_barang ditetapkan sebagai **Foreign Key** yang terhubung (*references*) ke kode_barang pada tabel barang untuk menjaga integritas relasi antar data.

```
CREATE TABLE penjualan (
  no_transaksi CHAR(5) PRIMARY KEY,
  tanggal DATE NOT NULL,
  kode_barang CHAR(5) NOT NULL,
  jumlah INT NOT NULL,
  FOREIGN KEY (kode_barang) REFERENCES barang(kode_barang) ON DELETE CASCADE ON UPDATE CASCADE
);
```

4. Input Data Awal

INSERT INTO digunakan untuk memasukkan daftar dataset awal produk dan transaksi penjualan sesuai dengan baris data pada instruksi soal.

```
INSERT INTO barang (kode_barang, nama_barang, kategori, harga, stok) VALUES
('BR001', 'Kertas A4', 'ATK', 65000, 120),
('BR002', 'Tinta Printer', 'Elektronik', 85000, 45),
('BR003', 'Pulpen', 'ATK', 2500, 500),
('BR004', 'Toner Printer', 'Elektronik', 450000, 20),
('BR005', 'Map Arsip', 'ATK', 5000, 200),
('BR006', 'Kalkulator', 'Elektronik', 125000, 35),
('BR007', 'Buku Agenda', 'ATK', 18000, 80),
('BR008', 'Label Pengiriman', 'Kemasan', 12000, 150);
```

```
INSERT INTO penjualan (no_transaksi, tanggal, kode_barang, jumlah) VALUES
('TR001', '2026-10-01', 'BR001', 10),
('TR002', '2026-10-02', 'BR002', 5),
('TR003', '2026-10-03', 'BR003', 50),
('TR004', '2026-10-04', 'BR005', 20),
```

```
('TR005', '2026-10-05', 'BR007', 15);
```

Tugas 2 – Membuat Stored Procedures

1. **Konsep Delimiter**

   Digunakan perintah DELIMITER $$ untuk mengganti pemisah baris sementara agar MariaDB tidak langsung mengeksekusi sintaks saat menemukan tanda titik koma (;) di dalam instruksi Stored Procedure.

   Diakhiri dengan DELIMITER ; untuk mengembalikan pemisah baris ke kondisi standar.

2. **Rincian Prosedur yang Dibuat**

   sp_tampil_barang(): Prosedur tanpa parameter yang menjalankan perintah SELECT * FROM barang; untuk membaca seluruh isi tabel barang.

```
DELIMITER $$
CREATE PROCEDURE sp_tampil_barang()
BEGIN
  SELECT * FROM barang;
END$$
DELIMITER ;
```

   sp_cari_kategori(IN p_kategori ...): Menerima masukan (*parameter IN*) berupa nama kategori, lalu menyaring baris barang dengan kondisi WHERE kategori = p_kategori.

```
DELIMITER $$
CREATE PROCEDURE sp_cari_kategori(
  IN p_kategori VARCHAR(30)
)
BEGIN
  SELECT * FROM barang WHERE kategori = p_kategori;
END$$
```

```
DELIMITER ;
```

sp_tambah_barang(...): Menerima 5 parameter data produk baru, kemudian mengeksekusi perintah INSERT INTO barang untuk menyimpan data tersebut ke basis data.

```
DELIMITER $$
CREATE PROCEDURE sp_tambah_barang(
  IN p_kode CHAR(5),
  IN p_nama VARCHAR(50),
  IN p_kategori VARCHAR(30),
  IN p_harga INT,
  IN p_stok INT
)
BEGIN
  INSERT INTO barang (kode_barang, nama_barang, kategori, harga, stok)
  VALUES (p_kode, p_nama, p_kategori, p_harga, p_stok);

  SELECT * FROM barang WHERE kode_barang = p_kode;
END$$
DELIMITER ;
```

sp_catat_penjualan(...): Menerima 4 parameter transaksi penjualan, lalu mengeksekusi INSERT INTO penjualan.

```
DELIMITER $$
CREATE PROCEDURE sp_catat_penjualan(
  IN p_no_tr CHAR(5),
  IN p_tgl DATE,
  IN p_kode CHAR(5),
```

```
    IN p_jumlah INT
)
BEGIN
    INSERT INTO penjualan (no_transaksi, tanggal, kode_barang, jumlah)
    VALUES (p_no_tr, p_tgl, p_kode, p_jumlah);

    SELECT * FROM penjualan WHERE no_transaksi = p_no_tr;
END$$
DELIMITER ;
```

Tugas 3 – Memanggil Stored Procedures

1. **Ekseskusi Pengujian**

- Sintaks CALL digunakan untuk memanggil dan menjalankan Stored Procedure yang sudah dibuat.

```
CALL sp_tampil_barang();
```

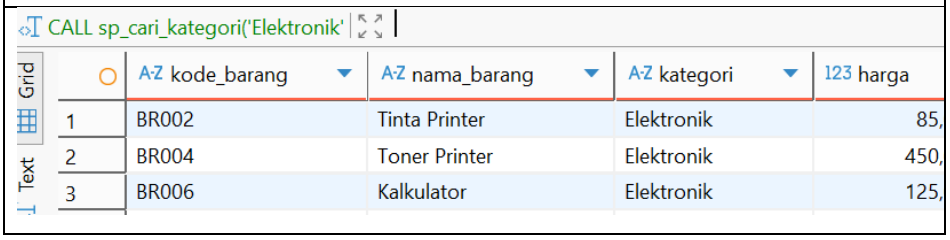
CALL sp_tampil_barang()

Grid | Text

	kode_barang	nama_barang	kategori	harga
1	BR001	Kertas A4	ATK	65,
2	BR002	Tinta Printer	Elektronik	85,
3	BR003	Pulpen	ATK	2,
4	BR004	Toner Printer	Elektronik	450,
5	BR005	Map Arsip	ATK	5,
6	BR006	Kalkulator	Elektronik	125,
7	BR007	Buku Agenda	ATK	18,
8	BR008	Label Pengiriman	Kemasan	12,

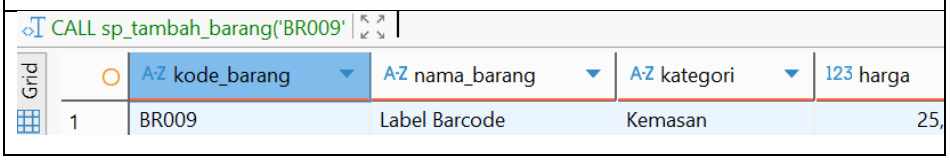
- Sintaks CALL digunakan untuk memanggil dan menjalankan Stored Procedure yang sudah dibuat.
- Menjalankan CALL sp_cari_kategori('Elektronik'); untuk menguji penyaringan data khusus produk kategori Elektronik.’

```
CALL sp_cari_kategori('Elektronik');
```



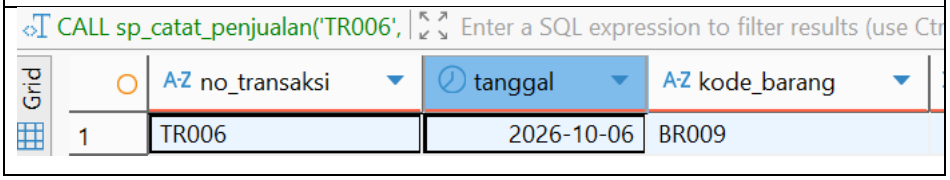
- Menjalankan CALL sp_tambah_barang(...) dengan menyisipkan nilai data BR009 (Label Barcode).

```
CALL sp_tambah_barang('BR009', 'Label Barcode', 'Kemasan', 25000, 75);
```



- Menjalankan CALL sp_catat_penjualan(...) dengan menyisipkan transaksi TR006.

```
CALL sp_catat_penjualan('TR006', '2026-10-06', 'BR009', 10);
```



2. **Verifikasi Data**

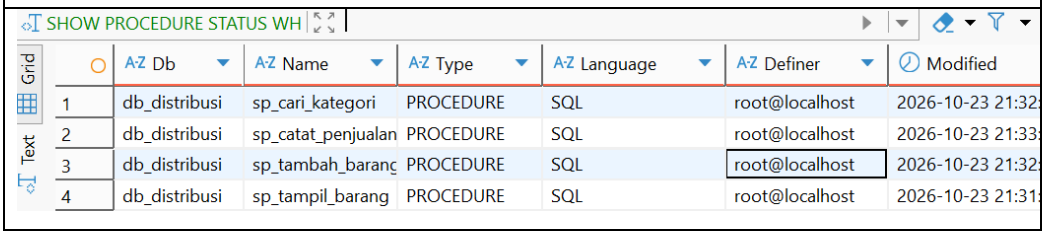
   Perintah SELECT dijalankan pada tabel barang dan penjualan untuk memastikan bahwa data yang diinput via Stored Procedure telah tersimpan secara permanen pada tabel target.

Tugas 4 – Menampilkan Daftar Stored Procedures

1. **Pemeriksaan Status**

Perintah SHOW PROCEDURE STATUS WHERE Db = 'db_distribusi'; mengeksekusi pencarian metadata pada sistem MariaDB untuk menampilkan daftar prosedur yang aktif pada database db_distribusi.

```
SHOW PROCEDURE STATUS WHERE Db = 'db_distribusi';
```



	Db	Name	Type	Language	Definer	Modified
1	db_distribusi	sp_cari_kategori	PROCEDURE	SQL	root@localhost	2026-10-23 21:32:
2	db_distribusi	sp_catat_penjualan	PROCEDURE	SQL	root@localhost	2026-10-23 21:33:
3	db_distribusi	sp_tambah_barang	PROCEDURE	SQL	root@localhost	2026-10-23 21:32:
4	db_distribusi	sp_tampil_barang	PROCEDURE	SQL	root@localhost	2026-10-23 21:31:

2. **Analisis dan Manfaat Bagi DBA**
    - Inventarisasi Sistem: Memudahkan DBA mengidentifikasi seluruh fungsi tersimpan (*stored logic*) yang berjalan di dalam database.
    - **Pemeliharaan & Keamanan:** Membantu DBA mengontrol hak akses pengguna serta menghindari duplikasi nama prosedur saat pengembangan sistem.

Tugas 5 – Mengubah Stored Procedures

1. **Mekanisme Perubahan Prosedure**

    Sintaks ALTER PROCEDURE pada MariaDB tidak mendukung pembaruan struktur logika query di dalam blok BEGIN ... END.

    Logika baru diterapkan dengan cara menghapus prosedur lama terlebih dahulu menggunakan DROP PROCEDURE IF EXISTS sp_tampil_barang;.

```
DROP PROCEDURE IF EXISTS sp_tampil_barang;
```

2. **Penerapan Logika Baru**

    Prosedur sp_tampil_barang dibuat ulang dengan penambahan klausul filter WHERE stok < 50 agar sistem hanya menampilkan barang yang membutuhkan re-stock (stok rendah).

    Saat diuji dengan CALL sp_tampil_barang();, daftar yang keluar hanya produk dengan kuantitas di bawah 50 unit.

```
DELIMITER $$
CREATE PROCEDURE sp_tampil_barang()
BEGIN
  SELECT * FROM barang WHERE stok < 50;
END$$
DELIMITER ;
```

```
CALL sp_tampil_barang();
```

CALL sp_tampil_barang() | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	kode_barang	nama_barang	kategori	harga	st
1	BR002	Tinta Printer	Elektronik	85,000	
2	BR004	Toner Printer	Elektronik	450,000	
3	BR006	Kalkulator	Elektronik	125,000	

Tugas 6 – Menghapus Stored Procedures

1. **Penghapusan Prosedur**

   DROP PROCEDURE sp_catat_penjualan; menghapus definisi prosedur mencatat penjualan.

   DROP PROCEDURE IF EXISTS sp_tambah_barang; menghapus prosedur penambahan barang secara aman tanpa menimbulkan kegagalan sistem jika nama prosedur tidak ditemukan.

```
DROP PROCEDURE sp_catat_penjualan;
```

```
DROP PROCEDURE IF EXISTS sp_tambah_barang;
```

2. **Pengujian Pesan Kesalahan**

   Memanggil prosedur yang telah dihapus (CALL sp_tambah_barang(...)) akan memicu error 1305 dari MariaDB karena definisi prosedurnya sudah tidak ada di dalam database.

```
SHOW PROCEDURE STATUS WHERE Db = 'db_distribusi';
```

SHOW PROCEDURE STATUS WH

Grid

	Db	Name	Type	Language	Definer	Modified
1	db_distribusi	sp_cari_kategori	PROCEDURE	SQL	root@localhost	2026-10-23 21:32:
2	db_distribusi	sp_tampil_barang	PROCEDURE	SQL	root@localhost	2026-10-23 21:48:

```
CALL sp_catat_penjualan('TR007', '2026-10-07', 'BR001', 5);
```

3. Pemeriksaan Keutuhan Data

Mengakses tabel barang dan penjualan membuktikan bahwa penghapusan Stored Procedure hanya menghapus perintah tersimpannya saja, sedangkan data transaksi dan produk di dalam tabel tetap aman dan tidak terhapus.

Bagaimana cara menambahkan penanganan error (error handling) pada Stored Procedure sp_catat_penjualan jika stok barang tidak mencukupi? Apa perbedaan antara Stored Procedure dan Stored Function pada MariaDB?

Penanganan Error Stok pada sp_catat_penjualan: Prosedur menggunakan perintah DECLARE untuk mengecek jumlah stok tersisa. Jika permintaan melebihi stok, prosedur membatalkan proses dengan SIGNAL SQLSTATE '45000' dan memunculkan pesan pesan *error*. Jika stok cukup, transaksi dicatat via INSERT dan stok dipotong otomatis dengan UPDATE.

Perbedaan Stored Procedure vs Stored Function:

Stored Procedure: Dipanggil dengan CALL, tidak wajib mengembalikan nilai, dan dipakai untuk eksekusi alur proses bisnis yang kompleks (termasuk INSERT, UPDATE, DELETE).

Stored Function: Dipanggil di dalam perintah SQL (seperti SELECT), wajib mengembalikan satu nilai (*return value*), dan dipakai untuk kalkulasi data.

```
SELECT * FROM barang;
```

SELECT * FROM barang

Grid | Text

	kode_barang	nama_barang	kategori	harga	123 s
1	BR001	Kertas A4	ATK	65,000	
2	BR002	Tinta Printer	Elektronik	85,000	
3	BR003	Pulpen	ATK	2,500	
4	BR004	Toner Printer	Elektronik	450,000	
5	BR005	Map Arsip	ATK	5,000	
6	BR006	Kalkulator	Elektronik	125,000	
7	BR007	Buku Agenda	ATK	18,000	
8	BR008	Label Pengiriman	Kemasan	12,000	
9	BR009	Label Barcode	Kemasan	25,000	

SELECT * FROM penjualan;

SELECT * FROM penjualan | Enter a SQL expression to filter results (use Ctrl+Space)

Grid

	no_transaksi	tanggal	kode_barang	123 jumla
1	TR006	2026-10-06	BR009	

Kesimpulan

Penggunaan *Stored Procedure* pada MariaDB terbukti meningkatkan efisiensi dan keamanan pengelolaan database db_distribusi, karena kumpulan perintah SQL seperti menampilkan data, menyaring kategori, menambah barang, dan mencatat penjualan dapat disimpan dan dipanggil kembali sewaktu-waktu menggunakan perintah CALL tanpa perlu menulis ulang query dari awal. Melalui praktikum ini juga dipahami bahwa perubahan logika perintah pada prosedur harus dilakukan dengan menghapus (DROP) dan membuat ulang (CREATE) prosedur baru karena keterbatasan sintaks ALTER PROCEDURE di MariaDB, serta dikonfirmasi bahwa pengelolaan atau penghapusan definisi *Stored Procedure* sama sekali tidak memengaruhi maupun merusak data fisik yang tersimpan di dalam tabel barang dan penjualan.